Adventist University of Central Africa
P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

Department of Big Data Analytics

Course: Big Data Analytic

Lecturer: Temitope Oguntade

FINAL EXAM REPORT

Submission Date: 30th Jan 2026

MPANO RUDAKENGA Rongin

ID: 101032

CHAP 1: INTRODUCTION

The rapid growth of e-commerce platforms has led to the generation of large volumes of heterogeneous data, including user browsing behavior, transaction records, and session-level interaction logs. Traditional single-database systems often struggle to efficiently store and analyze diverse data at scale, prompting modern big data solutions to adopt polyglot persistence, where multiple data storage technologies are combined to leverage their respective strengths.

This project presents the design and implementation of a Big Data Analytics system for an e-commerce platform using MongoDB, Apache HBase, and Apache Spark. MongoDB stores semi-structured transactional and product data, HBase manages high-volume session and time-series behavioral data, and Apache Spark performs distributed batch processing and advanced analytics across datasets.

The primary objective of the project is to demonstrate how multiple big data technologies can be integrated to extract meaningful insights from large-scale e-commerce data. The system supports customer behavior analysis, conversion funnel evaluation, Customer Lifetime Value (CLV) analysis and segmentation, and visualization of key business insights.

By combining distributed storage and processing frameworks, the project illustrates a scalable and flexible architecture aligned with core big data principles such as horizontal scalability, fault tolerance, and efficient analytical processing.

1.1 Project Scope and Objectives

This project focuses on batch-oriented analytics using synthetic e-commerce datasets designed to simulate real-world user behavior. The emphasis is on analytical workloads rather than real-time transaction processing. The main objectives are to:

1. Design and implement an integrated big data architecture using MongoDB, HBase, and Apache Spark.
2. Model and store e-commerce data based on structure and access patterns.
3. Perform large-scale analytics using Spark DataFrames and Spark SQL.
4. Derive actionable business insights such as conversion rates and customer lifetime value.
5. Visualize analytical results in a clear and interpretable manner.

1.2 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents the system architecture and technology interactions. Chapter 3 discusses data modeling decisions and storage rationale. Chapter 4 explains the Spark processing pipelines and analytical methodologies. Chapter 5 presents key findings and visualizations, while Chapter 6 discusses scalability considerations, limitations, and future improvements.

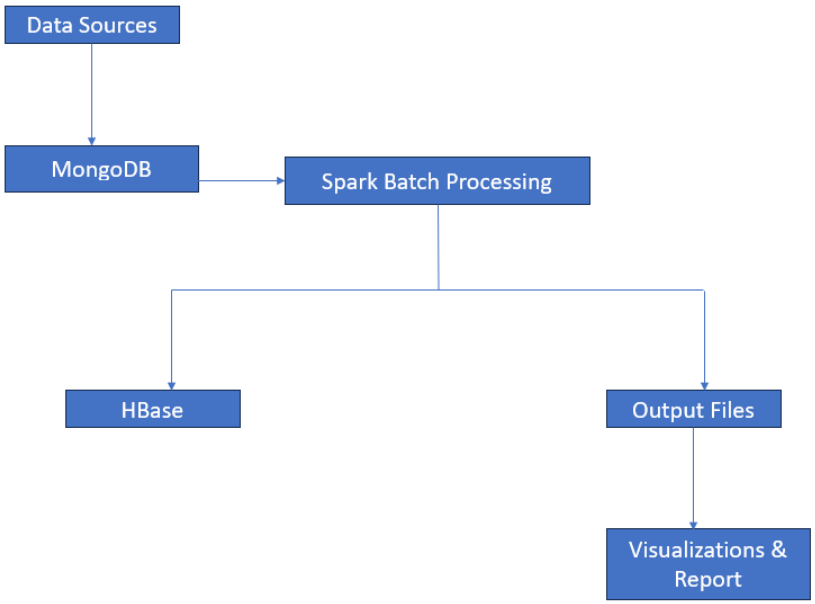
CHAP 2: SYSTEM ARCHITECTURE OVERVIEW

This project adopts a multi-tier big data architecture that integrates multiple data storage and processing technologies, selected according to data characteristics and analytical requirements. The architecture follows the principle of polyglot persistence, where different data models coexist to optimize performance, scalability, and analytical flexibility.

At a high level, the system consists of four layers:

1. Data Generation Layer
2. Data Storage Layer
3. Data Processing and Analytics Layer
4. Visualization and Insight Layer

Each layer is described in the following sections.



2.1 Data Generation Layer

The data generation layer produces synthetic e-commerce datasets that simulate real-world platform behavior. Generated data includes user sessions and browsing activity, product catalogs, and transaction and order data.

Session data captures detailed user interactions such as page views, device information, geographic location, and conversion status. Due to its high volume and time-series nature, session data is generated in bulk and stored as JSON files (e.g., sessions_0.json). This layer provides realistic and scalable datasets suitable for demonstrating big data storage and analytical techniques.

2.2 Data Storage Layer

The data storage layer uses multiple database technologies, each chosen based on the structure and access patterns of the data.

2.2.1 MongoDB for Transactional and Product Data

MongoDB stores product information, transaction records, and customer-related transactional summaries. These datasets are semi-structured and naturally represented as documents.

MongoDB's flexible schema supports nested structures such as product lists within transactions, simplifying storage and querying. This makes MongoDB well-suited for analytical tasks, including revenue aggregation, product performance analysis, and Customer Lifetime Value (CLV) computation.

2.2.2 HBase for Session and Behavioral Data

Apache HBase is used to store high-volume session-level browsing data. Each session includes identifiers, timestamps, device and geographic information, conversion status, and the full raw session JSON.

Sessions are stored using a wide-column data model with a composite row key combining user identity and session attributes. This design enables efficient scans and aggregations across large datasets. HBase's horizontal scalability and support for sparse data make it suitable for storing and analyzing tens of thousands of session records.

2.3 Data Processing and Analytics Layer

Analytical processing is performed using Apache Spark, which acts as the unified processing engine across data sources.

Spark is used to load transactional data, process batch-oriented datasets using DataFrames and Spark SQL, compute analytical metrics such as Customer Lifetime Value (CLV), and aggregate session data for behavioral and funnel analysis. Its distributed processing model enables efficient analysis of large datasets that would be impractical to process on a single node.

2.4 Visualization and Insight Layer

The visualization layer transforms analytical results into interpretable insights. After processing using Spark or Python-based analytics, results are exported as CSV or Parquet files and visualized using charts and plots.

Spark jobs generate aggregated datasets that are consumed by Python scripts to produce publication-ready charts, such as revenue trends, order status distribution, and CLV distribution, stored under the outputs/analysis_plots.py/ directory. These visualizations support a clear interpretation of user behavior and system performance.

2.5 Architecture Summary

In summary, the system architecture integrates:

- MongoDB for document-based storage of transactional data,
- HBase for scalable storage of session and behavioral data,
- Apache Spark for distributed batch analytics,
- Visualization tools for presenting analytical insights.

This layered architecture ensures scalability, modularity, and efficient analytical processing while clearly separating data ingestion, storage, processing, and visualization concerns.

CHAP 3: DATA MODELING DECISIONS AND RATIONALE

Efficient data modeling is a critical component of any big data system, as it directly affects storage efficiency, query performance, and scalability. In this project, different data models were selected based on dataset structure, access patterns, and analytical requirements. This chapter explains the rationale behind the data modeling decisions for MongoDB and Apache HBase.

3.1 Overview of Data Characteristics

The e-commerce datasets used in this project exhibit heterogeneous characteristics:

- Transactional data (orders, payments, purchased products) is semi-structured and naturally grouped by transaction.
- Session and behavioral data is high-volume, time-oriented, and sparse, with varying attributes across sessions.
- Analytical outputs require aggregation, filtering, and joining across datasets.

Due to these differences, a single data model would be inefficient. Therefore, a polyglot persistence approach was adopted.

3.2 MongoDB Data Model (Document-Oriented)

MongoDB was selected to store transactional and product-related data due to its flexible document-based model.

3.2.1 Modeled Entities

The main entities stored in MongoDB include products, transactions, and customer-related purchase summaries. Each transaction document embeds related information such as purchased products, quantities, and prices, reducing the need for costly joins and supporting common analytical queries.

3.2.2 Rationale for Using MongoDB

MongoDB was chosen because it provides schema flexibility, supports efficient aggregation operations for metrics such as total revenue and Customer Lifetime Value (CLV), and offers a natural representation of transactional data using JSON-like documents. This model integrates well with Spark-based batch analytics through DataFrames.

3.3 HBase Data Model (Wide-Column)

Apache HBase was selected to store user session and browsing behavior data, which is large in volume and time-dependent.

3.3.1 Table and Column Family Design

The project uses a single HBase table named session with one column family:

Table: session

Column Family: cf

```
def make_rowkey(obj: dict) -> str:
    """
    Strong rowkey:
    user_id#start_time#session_id
    (good for scans by user and time)
    """
```

```
def flatten_to_hbase_columns(obj: dict) -> dict:
    """
    Map your JSON into cf:* columns (bytes).
    Keep it simple + useful for analytics.
    """
    geo = obj.get("geo_data", {}) or {}
    dev = obj.get("device_profile", {}) or {}
    viewed_products = obj.get("viewed_products", []) or []
    page_views = obj.get("page_views", []) or []
    cart = obj.get("cart_contents", {}) or {}

    columns = {
        f"{COLUMN_FAMILY}:session_id": safe_str(obj.get("session_id")),
        f"{COLUMN_FAMILY}:user_id": safe_str(obj.get("user_id")),
        f"{COLUMN_FAMILY}:start_time": safe_str(obj.get("start_time")),
        f"{COLUMN_FAMILY}:end_time": safe_str(obj.get("end_time")),
        f"{COLUMN_FAMILY}:duration_seconds": safe_str(obj.get("duration_seconds")),
        f"{COLUMN_FAMILY}:conversion_status": safe_str(obj.get("conversion_status")),
        f"{COLUMN_FAMILY}:referrer": safe_str(obj.get("referrer")),
```

Each row represents a unique user session. Columns within the cf family store attributes such as:

- User identifiers,
- Session timestamps,
- Device and geographic metadata,
- Conversion status,
- Raw session JSON data.

This wide-column approach allows efficient storage of sparse attributes without enforcing a rigid schema.

3.3.2 Row Key Design

A composite row key was designed as:

user_id#start_time#session_id

This design ensures row-level uniqueness, supports efficient scans by user and time range, and aligns with user-centric analytical access patterns.

3.3.3 Rationale for Using HBase

HBase was selected due to its horizontal scalability, high write throughput for bulk session ingestion, flexible schema design, and suitability for time-series and behavioral data analysis.

3.4 Data Redundancy and Optimization Considerations

To balance performance and analytical flexibility, key session attributes are stored as individual columns for fast access, while the full raw session JSON is preserved to support future reprocessing or deeper analysis. This hybrid approach minimizes repeated parsing while maintaining extensibility.

3.5 Summary of Data Modeling Choices

In summary:

- MongoDB supports structured and semi-structured transactional data requiring aggregation and joins.
- HBase supports high-volume, time-series session data requiring scalable storage and fast scans.
- These data models align with analytical workflows implemented using Apache Spark.

Together, these modeling decisions ensure efficient storage, scalable analytics, and flexibility for future system extensions.

CHAP 4. SPARK PROCESSING PIPELINES AND METHODOLOGY

Apache Spark serves as the core analytical engine in this project, enabling scalable batch processing of large e-commerce datasets. Spark was chosen due to its in-memory computation model, support for structured data processing, and ability to integrate with multiple data sources such as MongoDB and file-based storage systems.

This chapter describes the Spark processing pipelines implemented in the project, the methodologies used for key analyses, and how Spark SQL and DataFrames were leveraged to derive business insights.

4.1 Spark Environment and Processing Mode

The project employs Spark batch processing, which is well suited for offline analytical workloads involving large datasets. All Spark jobs were executed using the DataFrame API and Spark SQL, providing scalable and expressive analytical capabilities.

This batch-oriented approach enables efficient processing of historical data, repeatable analytics workflows, and a clear separation between data ingestion and analysis. Spark jobs were implemented as modular Python scripts, each addressing a specific analytical task.

4.2 Data Ingestion into Spark

Spark ingests data from multiple sources depending on the analysis, including CSV and Parquet files from preprocessing steps, aggregated transactional data, and outputs from earlier Spark jobs.

Parquet is used extensively due to its columnar storage format, reduced I/O overhead, and improved performance for analytical queries. Once ingested, data is represented as Spark DataFrames, enabling efficient transformations and aggregations.

4.3 Customer Lifetime Value (CLV) Computation

Customer Lifetime Value (CLV) computation is a key analytical task in this project, representing the total revenue generated by a customer over the observed period.

CLV is computed by loading transactional data into Spark DataFrames, grouping transactions by customer identifier, aggregating total spending per customer, and calculating cumulative transaction values. Spark SQL and aggregation functions enable efficient computation across thousands of customer records. The resulting CLV metrics are stored in both CSV and Parquet formats for downstream analysis and visualization.

```
top10_clv = df.sort_values("clv", ascending=False).head(10)

print("\nTop 10 Customers by CLV:")
print(top10_clv[["user_id", "clv", "total_orders", "total_sessions"]])
```

4.4 Spark SQL for Analytical Queries

Spark SQL is used to perform analytical queries on structured datasets, including aggregation of daily sales and revenue trends, identification of top-performing products, and segmentation of customers based on CLV ranges.

By expressing analytical logic declaratively, Spark SQL improves readability, maintainability, and consistency of the analytical pipelines.

```
q_order_status_summary = """
SELECT status, COUNT(*) AS orders_count, SUM(total) AS revenue
FROM transactions
GROUP BY status
ORDER BY orders_count DESC
"""

daily_revenue = spark.sql(q_daily_revenue)
payment_method_revenue = spark.sql(q_payment_method)
top_products = spark.sql(q_top_products)
revenue_by_category = spark.sql(q_revenue_by_category)
order_status_summary = spark.sql(q_order_status_summary)
```

4.5 Analytical Output Generation

After processing, analytical outputs are written to disk in standardized formats. CSV files support inspection and reporting, while Parquet files provide efficient storage and reuse in future analyses. This separation between processing and visualization ensures modularity and reproducibility.

4.6 Methodology Summary

The Spark processing methodology emphasizes batch-oriented analytics for scalability, use of DataFrames and Spark SQL for structured processing, efficient storage formats, and clear separation of ingestion, processing, and visualization stages.

Execution Workflow (End-to-End).
The project follows a structured workflow: services are started using Docker Compose, raw JSON datasets are generated or verified, session data is ingested into HBase, Spark batch jobs are executed to generate analytical outputs, and plots and tables are produced from the results for reporting. This workflow ensures scalable, repeatable, and extensible analytics.

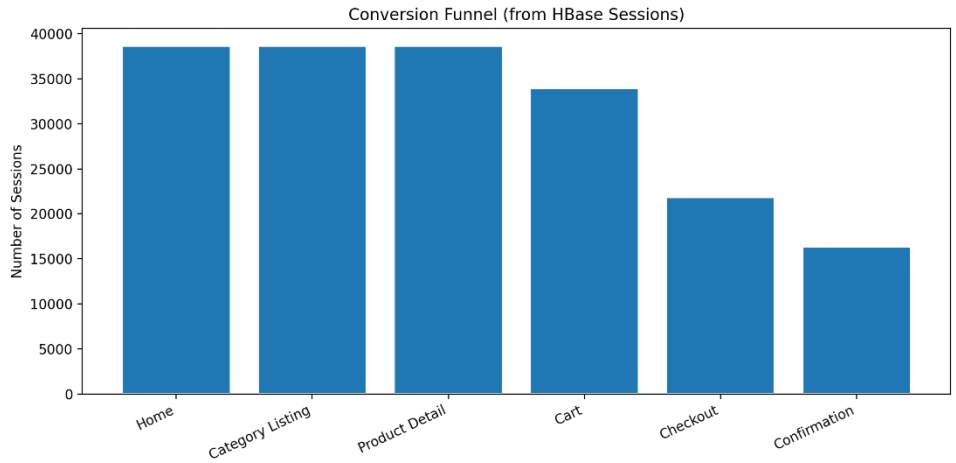
```
 Directory of C:\Users\mpalo\Documents\BIG DATA CLASS\BIG DATA ANALYTICS\Final\E_Commerce_Final_Project\outputs

26/01/2026  20:22    <DIR>          .
26/01/2026  20:27    <DIR>          ..
26/01/2026  20:40    <DIR>          analysis_plots
26/01/2026  10:59    <DIR>          analysis_tables
21/01/2026  14:47    <DIR>          batch_sessions
21/01/2026  15:23    <DIR>          final_integrated_outputs
18/01/2026  19:24             1 138 generator_log.txt
26/01/2026  10:52    <DIR>          hbase_exports
26/01/2026  20:22    <DIR>          spark_sql_analytics
               1 File(s)          1 138 bytes
               8 Dir(s)  77 537 886 208 bytes free

C:\Users\mpalo\Documents\BIG DATA CLASS\BIG DATA ANALYTICS\Final\E_Commerce_Final_Project\outputs>
```

CHAP 5: KEY FINDINGS AND BUSINESS INSIGHTS

5.1 Customer Conversion Behavior

Analysis of user session data stored in HBase reveals clear conversion patterns. Out of approximately 50,000 sessions, a large proportion of users exit the platform without completing a purchase. The conversion funnel shows a progressive drop-off from browsing to checkout, with the most significant decline occurring between the cart and checkout stages, indicating potential friction such as payment complexity or trust concerns.

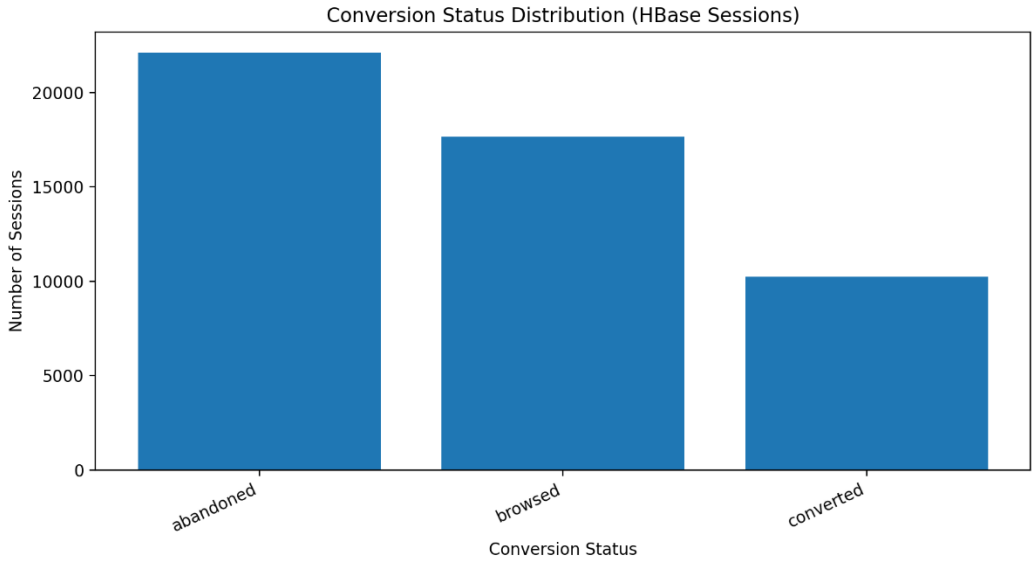
Only a small subset of sessions reaches the confirmation stage, highlighting the importance of optimizing late-stage user experience to improve overall conversion rates.



5.2 Conversion Status Distribution

Conversion status distribution further illustrates customer behavior trends. Most sessions are classified as abandoned, followed by browsed sessions, while converted sessions represent the smallest proportion. This imbalance indicates that although traffic volume is high, conversion efficiency remains limited.

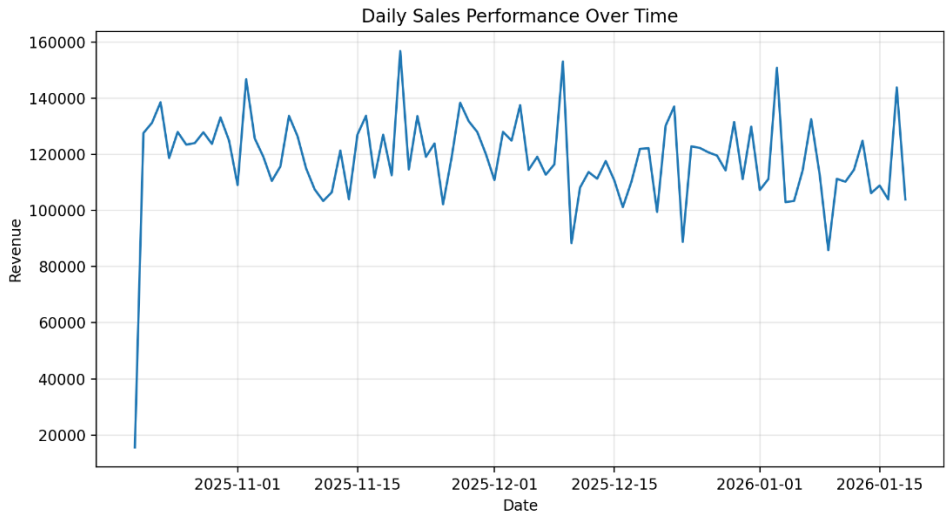
These insights support the need for targeted remarketing strategies and improved engagement mechanisms such as personalized offers and reminders.



5.3 Revenue Performance Over Time

Revenue analysis over time shows generally stable daily sales with noticeable fluctuations. Revenue peaks suggest the impact of promotional activities or seasonal demand, while dips may reflect reduced traffic or customer hesitation during certain periods.

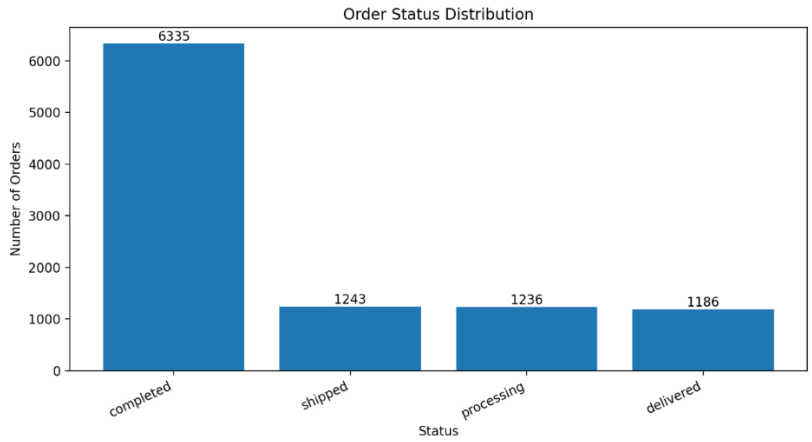
Monitoring revenue trends supports proactive marketing planning and helps identify optimal periods for promotions and inventory management.



5.4 Order Fulfillment and Revenue Contribution

Order status analysis shows that completed orders contribute the largest share of total revenue, while shipped, processing, and delivered orders account for smaller but comparable portions. This distribution reflects the operational flow of the e-commerce platform.

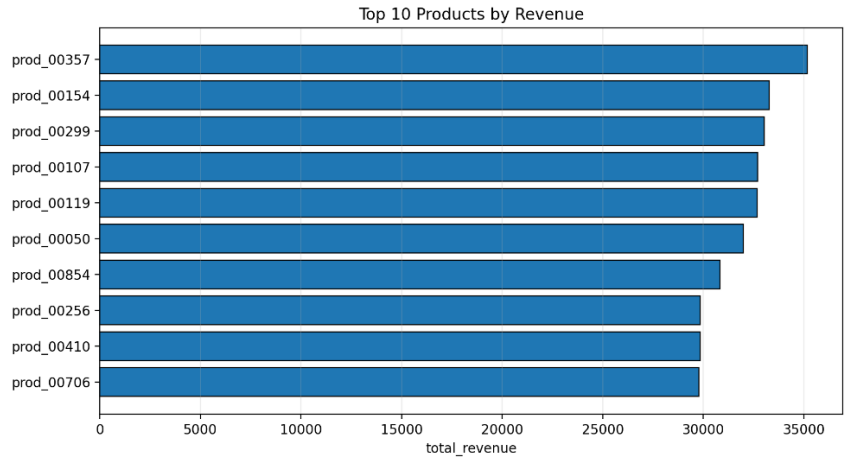
The dominance of completed orders as revenue drivers emphasizes the importance of minimizing order failures and processing delays to maximize revenue realization.



5.5 Top Products and Revenue Concentration

Product-level revenue analysis indicates strong revenue concentration, with a small group of top products generating a disproportionate share of total revenue. The top ten products consistently outperform others, suggesting focused demand.

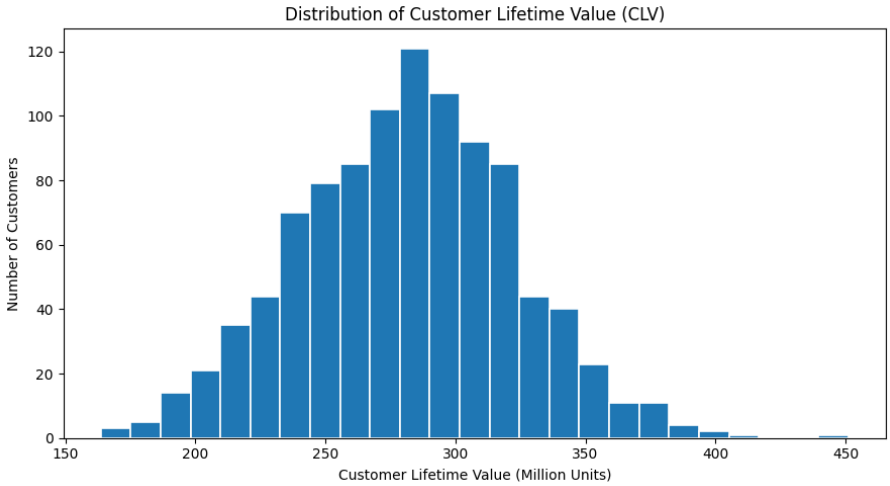
These findings support targeted inventory management, strategic pricing, and promotional efforts for high-performing products, while underperforming items may require repositioning or removal.



5.6 Customer Lifetime Value (CLV) Insights

CLV analysis reveals a wide distribution of customer value, with a small segment generating exceptionally high lifetime revenue. Customers can be segmented into low, medium, and high-value groups, enabling differentiated engagement strategies.

High-CLV customers represent long-term revenue potential and should be prioritized through loyalty programs and personalized experiences, while low-CLV customers present opportunities for upselling and engagement improvement.



5.7 Marketing Channel Effectiveness

Analysis of converted sessions by referrer shows that affiliate, email, and search engine channels are the most effective in driving conversions. Social and direct traffic also contribute, but at slightly lower levels.

These insights can guide marketing budget allocation by prioritizing high-performing channels and optimizing underperforming sources.

CHAP 6: SCALABILITY CONSIDERATIONS AND TECHNOLOGY SELECTION

6.1 Scalability Considerations

The proposed e-commerce analytics architecture was designed with scalability as a core requirement, considering both data volume growth and analytical complexity.

The system processes large-scale session data (50,000+ records) and transactional datasets while supporting analytical queries and batch processing.

HBase enables horizontal scalability by distributing session data across multiple region servers. As the number of user sessions increases, additional nodes can be added to the HBase cluster without disrupting existing operations. This design supports high-throughput writes, which is critical for ingesting real-time or near-real-time session data.

Apache Spark provides scalable batch analytics by distributing computations across a cluster. Spark's in-memory processing significantly improves performance for large aggregations such as conversion funnel analysis, customer lifetime value (CLV) computation, and revenue analytics. As data volume grows, Spark can scale by increasing executor memory or adding worker nodes.

MongoDB supports scalable document storage for semi-structured data, allowing flexible schema evolution as business requirements change. Its horizontal scaling via sharding makes it suitable for growing transactional datasets.

Overall, the separation of storage and processing layers ensures that the system can scale independently across ingestion, storage, and analytics workloads.

6.2 Technology Selection Justification

The e-commerce analytics architecture was designed with scalability as a core requirement, addressing both data volume growth and analytical complexity. The system processes large-scale session data (50,000+ records) and transactional datasets while supporting batch analytics.

HBase enables horizontal scalability by distributing session data across region servers, allowing additional nodes to be added as session volume grows. This design supports high-throughput writes required for ingesting large volumes of session data.

Apache Spark provides scalable batch analytics by distributing computations across a cluster. Its in-memory processing improves performance for large aggregations such as conversion funnel analysis, Customer Lifetime Value (CLV) computation, and revenue analytics. As data volume increases, Spark can scale by adding worker nodes or increasing executor resources.

MongoDB supports scalable document storage for semi-structured transactional data and allows flexible schema evolution as business requirements change. Horizontal scaling through sharding makes it suitable for growing transactional workloads.

Overall, the clear separation between storage and processing layers enables independent scaling across ingestion, storage, and analytics components.

6.3 Alignment with Analytical Tasks

Each technology in the architecture directly supports specific analytical objectives:

- MongoDB supports flexible storage and retrieval of transactional data for sales and order analysis.
- HBase enables efficient session tracking and behavioral analytics such as conversion funnels and referrer effectiveness.
- Spark integrates data from multiple sources and performs large-scale computations required for business insights.
- This division of responsibilities improves system performance, simplifies maintenance, and enhances analytical flexibility.

CHAP 7: METHODOLOGY, LIMITATIONS, AND FUTURE WORK

7.1 Methodology Explanation for Key Analyses

This project followed a structured big data analytics methodology consisting of data ingestion, storage, processing, analysis, and visualization. Each analytical task was designed to address specific business questions related to customer behavior and sales performance.

7.1.1 Customer Lifetime Value (CLV) Analysis

Customer Lifetime Value was computed using aggregated transactional data to measure total revenue generated by each customer over the observed period. Apache Spark was used to efficiently aggregate large datasets and compute CLV metrics at scale.

Customers were subsequently segmented into Low, Medium, and High CLV categories based on value thresholds derived from the CLV distribution.

7.1.2 Conversion Funnel Analysis

Session-level data stored in HBase was analyzed to construct a conversion funnel representing user progression through key stages, including browsing, product interaction, cart, checkout, and confirmation.

This analysis involved scanning HBase records, extracting session attributes, and counting the number of sessions reaching each stage, enabling identification of major drop-off points in the customer journey.

7.1.3 Conversion Status and Referrer Analysis

Session conversion status (abandoned, browsed, converted) and traffic referrers (direct, email, search engine, affiliate, social) were analyzed to evaluate marketing channel effectiveness.

Aggregations were performed using Spark, enabling efficient grouping and counting across tens of thousands of session records.

7.1.4 Sales and Revenue Analysis

Daily sales performance was analyzed to identify revenue trends over time. Order status and revenue were further examined to understand fulfillment progress and revenue contribution by order stage.

7.2 Experimental Constraints and Dataset Configuration

In this project, synthetic e-commerce data was generated using the Faker library to simulate realistic user behavior, transactions, and session activity. The dataset sizes were intentionally configured to balance analytical completeness with the computational limitations of the local development environment.

Due to hardware constraints of the personal computer used for experimentation (limited memory and processing capacity), the number of generated entities was reduced to manageable yet analytically meaningful scales, including 1,000 users, 1,000 products, 10,000 transactions, and 50,000 user sessions over a 90-day period. These values were selected to ensure stable execution of HBase ingestion, Spark batch processing, and visualization workflows without performance degradation or system failure.

Despite these controlled reductions, the dataset remains sufficiently large to demonstrate Big Data characteristics such as high volume session logs, distributed storage behavior, and scalable analytics using Apache Spark. In a production or cloud-based environment, the same data generation and processing pipelines could be executed with significantly larger datasets without modification to the system design.

CHAP 8: CONCLUSION

This project implemented an integrated Big Data analytics system for e-commerce using MongoDB, HBase, and Apache Spark to analyze large-scale transactional and session data.

A polyglot persistence architecture was applied, with MongoDB managing transactional data, HBase handling high-volume session behavior, and Apache Spark performing scalable batch analytics. This design enabled efficient storage, processing, and analysis of heterogeneous data.

Key analytics included Customer Lifetime Value (CLV) computation, customer segmentation, conversion funnel analysis, and revenue and order status evaluation. The results identified customer drop-off patterns, high-value customer segments, and effective traffic acquisition channels.

Overall, the project met the assignment requirements and demonstrated how modern Big Data technologies can be integrated to support scalable e-commerce analytics, providing a foundation for future extensions such as real-time analytics and predictive modeling.